ORPHE CORE SDK for Android

ORPHE CORE に接続するための Java SDK を提供します。

利用方針（ライセンス）

本 SDK は、ORPHE Inc. が定める利用方針のもとで提供するソースコード公開（source-available）のソフトウェアです。正式な条文は LICENSE.md を参照してください。

- **無償で利用できる用途**：教育・ワークショップ、非商用研究、個人制作、試作・社内検証（企業における製品・技術の評価を含む）、無償アプリ・非商用サービス
- **別途 ORPHE との商用契約が必要な用途**：有料アプリ・有料サービス、商用 SDK 連携、受託開発、事業として提供するサービス、本 SDK を組み込んだ製品・サービスの第三者への提供

商用契約のご相談: https://orphe.io/

インストール

aar ファイルを追加

※開発途中であり aar ファイルが最新でない場合があります。その場合は上記の**ソースそのものを利用する**で SDK を追加してください。

1. `orphecoresdk-release-xxx.aar` を下記の URL に従いインポートします。
   https://zenn.dev/apple_nktn/articles/d6f7e0cac8d413

サンプルアプリの使い方

サンプルアプリでは、ORPHE INSOLE をスキャンして接続し、センサー値を計測できます。計測結果はアプリ内の履歴へ保存され、左右のデータを CSV ファイルとして端末へ保存または他のアプリへ共有できます。

1. スキャンする

1. 端末の Bluetooth を有効にしてサンプルアプリを起動します。
2. 初回起動時に Bluetooth のスキャン権限を求められた場合は許可します。Android 11 以前では位置情報の権限も必要です。
3. ORPHE INSOLE のリセットボタンを押して光らせ、アドバタイズ状態にします。
4. `Scan Devices` を押します。
5. Left または Right の欄にデバイス ID と充電状態が表示されるまで待ちます。見つからない場合は `Scan Again` を押して再試行します。

2. 接続する

1. 接続する側の `Left Connect` または `Right Connect` を押します。
2. 同じ側に複数のデバイスが見つかった場合は、表示されたデバイス ID と充電状態を確認して接続先を選択します。
3.「機器に接続されました」と表示されたことを確認します。

左右両方を接続することも、片側だけを接続して計測することもできます。接続を解除する場合は `Left Disconnect` または `Right Disconnect` を押します。

3. 計測する

1. 必要に応じて、画面上部で受信方式とサンプリングレートを選択します。

- Realtime: デバイスからリアルタイムにセンサー値を受信します。100Hz または 200Hz を選択できます。
- FIFO: 欠損したデータを可能な限り回収しながら受信します。サンプリングレートは 200Hz に固定されます。
- Quaternion グラフは Realtime かつ 100Hz を選択した場合のみ表示されます。デバイス側 (0x38) が返すクオータニオンのみ動作検証済みのため、他の組み合わせでは表示しません。

2. Start を押して計測を開始します。
3. センサー値のグラフと、Measuring / Left: ... / Right: ...に表示される左右の受信件数を確認します。
4. 計測を終了する場合は Stop & Save を押します。計測が停止し、受信したデータが Measurement History へ保存されます。

Clear は、画面上のグラフと現在の計測データを消去します。すでに Measurement History へ保存された結果は削除されません。

4. 結果を CSV ファイルとして保存する

Stop & Save では計測結果がアプリ内の Measurement History へ保存されます。端末上の任意の場所へ CSV ファイルとして書き出す場合は、次の操作を行います。

1. Measurement History から対象の結果にある Detail を押します。
2. 計測日時、計測時間、左右の受信件数、受信方式、サンプリングレート、CSV のプレビューを確認します。
3. Save Left CSV または Save Right CSV を押します。
4. Android のファイル保存画面で保存先とファイル名を指定します。

データが存在する側だけ保存ボタンが有効になります。履歴そのものを削除する場合は Delete を押します。

CSV のクオータニオン列（quatW〜quatZ）は、Realtime かつ 100Hz で計測した場合だけ出力されます。Request / FIFO、および Realtime の 200Hz では、これら 4 列はヘッダーを含めて出力されません。Euler 角は全モードで CSV へ出力されません。

5. 結果をシェアする

1. Measurement History にある対象結果の Share を押します。詳細画面の Share CSV からも同じ操作ができます。
2. Android の共有先選択画面から、CSV ファイルを渡すアプリを選択します。

片側だけを計測した場合は 1 つ、左右両方を計測した場合は左右の CSV ファイル 2 つが共有されます。

圧力補正値を設定する

- 左右のインソールへ接続すると、それぞれの Pressure Calibration ボタンが有効になります。
- 6 点の圧力センサーごとに coefficient1、coefficient2、coefficient3、threshold を入力し、Save を押すと受信中の圧力値へ即時反映されます。
- 設定はデバイス ID 別に端末へ保存され、同じデバイスへ再接続したときに自動で適用されます。
- Reset to defaults を押すと、そのデバイスの保存値を削除し、全 6 点を既定値へ戻します。

- サンプルアプリは補正値を自動推定しません。測定などで決定した補正値を手動入力するための機能です。

利用方法

事前準備

※app/src/main/java/io/orphe/orphecoresdkforandroid/MainActivity.java にサンプルコードが記載されています。

- SDK モジュールの Manifest には権限宣言が含まれていないため、利用側アプリの AndroidManifest.xml へ以下を追加します。

```
<!-- Android 12 以降の Bluetooth 権限 -->
<uses-permission
    android:name="android.permission.BLUETOOTH_SCAN"
    android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />

<!-- Android 11 以下の旧権限 -->
<uses-permission android:name="android.permission.BLUETOOTH"
android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"
android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"
android:maxSdkVersion="30" />
<uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION"
android:maxSdkVersion="30" />
```

BLUETOOTH_SCAN に android:usesPermissionFlags="neverForLocation"を付けないと、Android 12 以降で ACCESS_FINE_LOCATION の許可も必要になり、許可されていない場合はスキャン結果が無言で 0 件になります。

- Android 11 以下では、権限が許可されていても**端末の位置情報サービスが OFF だとスキャン結果が 0 件**になります。LocationManagerCompat.isLocationEnabled()で確認し、OFF の場合はユーザーへ案内してください。
- パーミッションの確認と必要であればリクエストを行います。Android のバージョンによって位置情報の権限が必要な場合と Bluetooth 系の権限が必要な場合があります。

```
private boolean hasBlePermissions() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        return checkSelfPermission(Manifest.permission.BLUETOOTH_SCAN)
                        == PackageManager.PERMISSION_GRANTED
                &&
checkSelfPermission(Manifest.permission.BLUETOOTH_CONNECT)
                        == PackageManager.PERMISSION_GRANTED;
    }
    return
checkSelfPermission(Manifest.permission.ACCESS_FINE_LOCATION)
```

```
            == PackageManager.PERMISSION_GRANTED;
}

private void requestBlePermissions() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        requestPermissions(new String[]{
                Manifest.permission.BLUETOOTH_CONNECT,
                Manifest.permission.BLUETOOTH_SCAN}, 0);
    } else {
        requestPermissions(new String[]{
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.BLUETOOTH_ADMIN,
                Manifest.permission.BLUETOOTH}, 0);
    }
}
```

ORPHE INSOLE の場合

- `OrpheInsoleCallback` オブジェクトを作成します。この中に ORPHE INSOLE の各イベントに対しての動作を記述します。

```
private final OrpheInsoleCallback mOrpheInsoleCallback = new
OrpheInsoleCallback() {
    @Override
    public void gotInsoleValues(OrpheInsoleValue[] values) {
        Log.d(TAG, values[0].toString());
    }

    @Override
    public void gotDeviceInfo(DeviceInfoValue value) {
        Log.d(TAG, value.batteryStatus.name());
    }

    @SuppressLint("MissingPermission")
    @Override
    public void onScan(BluetoothDevice bluetoothDevice, OrpheScanedMeta
meta) {
        if (mConnectionStatusTextViewLeft != null) {
            if (bluetoothDevice != null) {
                final String deviceId = meta.deviceId;
                Log.d(TAG, String.format("%s：機器が見つかりました",
deviceId));
            } else {
                Log.d(TAG, "機器が見つかりませんでした");
            }
        }
    }

    @SuppressLint("MissingPermission")
```

```
    @Override
    public void onConnect(BluetoothDevice bluetoothDevice) {
        if (mConnectionStatusTextViewLeft != null) {
            Log.d(TAG, String.format("%s：機器に接続されました",
bluetoothDevice.getName()));
        }
    }

    @SuppressLint("MissingPermission")
    @Override
    public void onDisconnect(BluetoothDevice bluetoothDevice) {
        if (mConnectionStatusTextViewLeft != null) {
            Log.d(TAG, String.format("%s：機器の接続が解除されました",
bluetoothDevice.getName()));
        }
    }
};
```

- 作成した OrpheInsoleCallback と対応する取り付け位置（OrpheSidePosition.leftPlantar など）を指定して OrpheInsole オブジェクトを作成します。

```
mOrpheInsole = new OrpheInsole(this, mOrpheInsoleCallbackLeft,
OrpheSidePosition.leftPlantar);
```

- 作成した OrpheInsole オブジェクトの startScan を呼び出します。

```
mOrpheInsole.startScan();
```

- ORPHE INSOLE のリセットボタンを押し光らせます。
  - 光った場合はアドバタイズしている状態になります。
- OrpheInsoleCallback の onScan に見つかったアドバタイズ中の ORPHE INSOLE の BluetoothDevice オブジェクトが渡されます。
- ORPHE INSOLE の判定は、**アドバタイズ名の INS プレフィックス**を最優先で行い、一致しない場合にメーカーデータのシグネチャで判定します。そのためメーカーデータを持たない世代（アドバタイズ名が INS0…の機体など）も検出できます。
  - メーカーデータが取得できる場合、OrpheScanedMeta の deviceId は従来どおり IN＋8 桁 16 進数＋L/R 形式になります。取得できない場合は**アドバタイズ名がそのまま deviceId** になります。deviceId を圧力補正値の保存キーに使う場合はこの違いに注意してください。
  - メーカーデータから左右が判別できない場合、OrpheScanedMeta の side は null（sideIsUnknown()が true）になり、**左右両方の OrpheInsole に候補として通知されます**。どちらへ接続するかは呼び出し側で選択してください。この挙動は setScanConfig に allowUnknownSideCandidate = false の OrpheInsoleScanConfig を渡すと無効化できます。

    - 左右不明のまま接続した場合、接続直後にデバイスから取得した左右と期待値が異なると `OrpheInsoleCallback` の `onSideMismatch` が呼ばれます（接続は維持されます）。
- スキャンの開始に失敗した場合は `OrpheInsoleCallback` の `onScanFailed` にエラーコードが渡されます。`6`（`SCANNING_TOO_FREQUENTLY`）は Android のスキャン頻度制限（アプリあたり 30 秒に 5 回）に達したことを示します。
- `OrpheInsoleCallback` の `onScan` で渡された `BluetoothDevice` を `OrpheInsole` オブジェクトの `connect` に渡すことで接続されます。

```
mOrpheInsole.connect(mBluetoothDevice);
```

    - `OrpheScanedMeta` の `deviceId` を参照することでデバイス ID を取得できます。
    - `OrpheScanedMeta` の `chargeStatus` を参照することで充電の状況（Wireless or Wired）。
- 接続後 `OrpheInsoleCallback` の `onConnect` のコールバックが呼び出されます。
- センサー値の取得に関して
    - `OrpheInsoleSensorConfig` を指定すると、接続後の初期設定として受信方式とサンプリングレートを選択できます。

```
OrpheInsoleSensorConfig config = new OrpheInsoleSensorConfig(
        OrpheSensorReceiveMode.realtime,
        OrpheInsoleSamplingRate.hz200
);
mOrpheInsole = new OrpheInsole(
        this,
        mOrpheInsoleCallback,
        OrpheSidePosition.leftPlantar,
        OrpheAccRange.range16,
        OrpheGyroRange.range2000,
        false,
        config
);
```

        - `OrpheSensorReceiveMode.realtime`: デバイスが送信したセンサー値をそのまま Notify で受け取ります。
        - `OrpheSensorReceiveMode.request`: アプリから指定した範囲だけを手動でリクエストします。
        - `OrpheSensorReceiveMode.fifo`: SDK が carry-over 方式で常時リクエストし、欠損検出・再要求・重複排除・全体値へのシリアル順マージまで自動で行います。受信値のコールバックは欠損回収を待たず即時に呼ばれます。
        - `OrpheInsoleSamplingRate.hz100`: 100Hz 出力。`realtime` ではクオータニオン付きの 0x38 を受信し、SDK がオイラー角を計算します。`request` / `fifo` では FW の 200Hz 蓄積データ (0x36) を SDK 内で姿勢計算してから 100Hz へ間引きます (未検証のため動作は保証しません)。
        - `OrpheInsoleSamplingRate.hz200`: 200Hz 出力。`realtime` ではクオータニオンを含まない 0x37 のため、クオータニオン・オイラー角は 0 です。`request` / `fifo` で

は FW の蓄積・再要求形式である 0x36 を受信し、SDK 内でクオータニオンとオイラー角を計算します（未検証のため動作は保証しません）。

`request` / `fifo` は Android 13（API 33）以上で利用できます。Android 8.0（API 26）から Android 12L（API 32）までは `realtime` を使用してください。

SDK 利用側で欠損を可能な限り回収しながら 100Hz 出力を得る場合は以下のように指定します。FW から受け取る 0x36 は 200Hz 固定で、SDK は 4 点すべてを 5ms 間隔で姿勢計算した後、0ms / 10ms の 2 点へ間引きます。アプリ側に姿勢計算用タイマーや再要求処理は不要です。

```
OrpheInsoleSensorConfig config = new OrpheInsoleSensorConfig(
        OrpheSensorReceiveMode.fifo,
        OrpheInsoleSamplingRate.hz100
);
```

FIFO の既定値は、200ms ごとに最大 200 シリアルを確認し、応答を最大 5 秒待ちます。Android 版では応答を 1 件以上受信した後に 250ms 無通信となった時点で要求を終了し、未着値を次回へ carry-over するため、部分的な BLE ロスのたびに 5 秒待ち続けません。FW から `noData` が返った値だけを回復不能として `OrpheInsoleCallback.sensorValueIsNotFound` へ通知します。carry-over が 100 シリアルを超えた場合、または最新値との差が 1,500 シリアルを超えた場合は、FWリングバッファの安全範囲へ再同期します。

FIFO では、値を受信するたびに `OrpheInsoleValueUpdate` 版のコールバックが呼ばれます。`getDeltaValues()`は今回受信した差分、`getAllValues()`はその時点までに SDK が取得・欠損回収できた全値です。欠損パケットが後から回収された場合は、その位置から後続のクオータニオンを時系列順に再計算します。全値はシリアル順にマージされ、実際に `getAllValues()`を呼んだ時だけ配列が生成されます。

```
OrpheInsoleCallback callback = new OrpheInsoleCallback() {
    @Override
    public void gotInsoleValues(OrpheInsoleValueUpdate update) {
        OrpheInsoleValue[] deltaValues = update.getDeltaValues();
        OrpheInsoleValue[] allValues = update.getAllValues();
    }
};
```

従来の `gotInsoleValues(OrpheInsoleValue[] values)`を実装している場合も変更は不要です。FIFO では同メソッドへ即時差分が渡されます。現在の全値だけを後から参照する場合は `OrpheInsole#getFifoValues()`も利用できます。

- `OrpheSensorReceiveMode.request` で手動取得する場合は、接続後にセンサー値送信のリクエストを送ります。1 範囲なら `requestInsoleValue(startSerialNumber, length)`、複数範囲なら `requestInsoleValue(OrpheValueRequest[])`を使用できます（最大 30 範囲）。

```
mOrpheInsole.requestInsoleValue(1200, 100);
```

  - `requestLatestInsoleValue` を呼び出すことで最新のセンサー値を取得することが可能です。

```
mOrpheInsole.requestLatestInsoleValue();
// もしくは
mOrpheInsole.requestLatestInsoleValue(100);
```

    - `length` のパラメーターを指定した場合は、**センサー値の最終取得時刻から予想されるシリアル番号から length 件**を取得します。
      - 取得件数が少なすぎると時間のズレ等でうまく取得できない可能性があるので 100 件程度までは数を増やしてください。
    - `length` のパラメータを指定しない場合は、**センサー値の最終取得時刻から予想されるシリアル番号から現在時刻までのセンサー値を取得**します。
    - 最新シリアル番号をまだ取得していない場合、最初の呼び出しでは現在のシリアル番号だけを要求して終了します。`OrpheInsoleCallback.gotCurrentSerialNumber` が呼ばれた後に、もう一度呼び出してください。
  - `requestInsoleValue` を呼び出すことで自由にデバイス内に一時保存されているセンサー値を取得することができます。
    - `OrpheValueRequest` に最初のシリアル番号とそこから取得する件数を指定してパラメータに渡してください。(最大 30 種類リクエストを送ることが可能)
  - 新しい手動リクエストはデバイス上の古いリクエストを置き換えます。常時取得には、要求を直列化する `fifo` を使用してください。
- リクエストされたセンサー値は 200Hz 形式の 0x36 として Notify で送信され、SDK がクオータニオンを計算してから `OrpheInsoleCallback` の `gotInsoleValues` に渡します。FIFO では欠損箇所以降も待機させず、受信できた値から即時に通知し、欠損回収後に SDK 内の後続値を再計算します。
  - `hz200` では、6 点の圧力センサー値を含むセンサー値を 1 シリアル番号につき 4 時点分取得します。`hz100` では SDK がその 4 時点を 2 時点へ間引きます。どちらもシリアル番号 1 増えるごとのインターバルは理論上 20ms です。
  - Euler 角の計算と公開フィールドは一時的に無効化されています。
  - Madgwick 6 軸フィルタは磁気センサーを使わないため、算出されるクオータニオンの yaw 方向には時間経過によるドリフトがあります。
  - `startTime`、`endTime`、`receivedAt` は epoch ミリ秒です。秒へ変換する場合は `1000.0` で割ります。
- 作成した `OrpheInsole` オブジェクトの `disconnect` を呼び出すことで切断します。

```
mOrpheInsole.disconnect();
```

- 作成した `OrpheInsole` オブジェクトの `status` で現在の接続ステータスを把握することが可能です。
- 作成した `OrpheInsole` オブジェクトの `getDeviceInfo` を呼び出すことでバッテリー情報を含む ORPHE INSOLE の情報を取得することができます。取得した値は `OrpheInsoleCallback` の `gotDeviceInfo` に渡されます。

```
mOrpheInsole.getDeviceInfo();
```

- リアルタイムモードへの切り替えは setSensorRequestMode を呼び出すことで可能です。
  - リセットされるともとに戻るため接続時に 1 度だけ実行することを推奨します。
  - OrpheInsoleSensorConfig を指定した場合は SDK が Notify 開始後に自動で切り替えます。
  - インソール（圧力込み）用の 200Hz リアルタイムモードは OrpheSensorRequestMode.realtimeForInsole、100Hz リアルタイムモードは OrpheSensorRequestMode.realtimeForInsoleWithQuaternion と指定します。

```
mOrpheInsole.setSensorRequestMode(OrpheSensorRequestMode.realtimeForInsole);
```

- 圧力の変換係数は setPressureCalibration メソッドで 6 点分を一括設定できます。各部位に coefficient1、coefficient2、coefficient3、threshold を設定できます。

```
OrpheInsolePressureCalibration calibration = new
OrpheInsolePressureCalibration(
        // toeInside
        new OrpheInsolePressureCoefficient(2.77942, 0.00235, 4.14411,
240.0),
        // midInside
        new OrpheInsolePressureCoefficient(2.70000, 0.00230, 4.10000,
235.0),
        // toeOutside
        new OrpheInsolePressureCoefficient(2.80000, 0.00240, 4.20000,
245.0),
        // center
        new OrpheInsolePressureCoefficient(2.60000, 0.00220, 4.00000,
230.0),
        // midOutside
        new OrpheInsolePressureCoefficient(2.90000, 0.00250, 4.30000,
250.0),
        // heel
        new OrpheInsolePressureCoefficient(3.00000, 0.00260, 4.40000,
255.0)
);
mOrpheInsole.setPressureCalibration(calibration);
```

  - OrpheInsolePressureCoefficient の 4 引数は順に coefficient1、coefficient2、coefficient3、threshold です。
  - 従来の 2 引数コンストラクタも利用でき、その場合は coefficient2 = 0.00235、threshold = 240mV が補完されます。
  - コンストラクタの部位順は toeInside、midInside、toeOutside、center、midOutside、heel です。
  - 既定値へ戻す場合は setPressureCalibration(OrpheInsolePressureCalibration.DEFAULT)を使用します。

- 従来の `setCoefficient(sensorPosition, coefficient, value)`も引き続き利用でき、現在の一括設定のうち指定した 1 項目だけを更新します。
- 計算式は `coefficient1 * exp(coefficient2 * milliVolt) + coefficient3` です。
- `milliVolt <= threshold` の場合は `0N` として扱います。
- 閾値以下、負の計算結果、または異常な入力値は `0N` として扱います。

ORPHE CORE の場合

- `OrpheCoreCallback` オブジェクトを作成します。この中に ORPHE CORE の各イベントに対しての動作を記述します。

```
private final OrpheCoreCallback mOrpheCoreCallback = new
OrpheCoreCallback() {
    @Override
    public void gotSensorValues(OrpheSensorValue[] values) {
        Log.d(TAG, values[0].toString());
    }

    @Override
    public void gotDeviceInfo(DeviceInfoValue value) {
        Log.d(TAG, value.batteryStatus.name());
    }

    @SuppressLint("MissingPermission")
    @Override
    public void onScan(BluetoothDevice bluetoothDevice, OrpheScanedMeta
meta) {
        if (mConnectionStatusTextViewLeft != null) {
            if (bluetoothDevice != null) {
                final String deviceId = meta.deviceId;
                Log.d(TAG, String.format("%s：機器が見つかりました",
deviceId));
            } else {
                Log.d(TAG, "機器が見つかりませんでした");
            }
        }
    }

    @SuppressLint("MissingPermission")
    @Override
    public void onConnect(BluetoothDevice bluetoothDevice) {
        if (mConnectionStatusTextViewLeft != null) {
            Log.d(TAG, String.format("%s：機器に接続されました",
bluetoothDevice.getName()));
        }
    }

    @SuppressLint("MissingPermission")
    @Override
    public void onDisconnect(BluetoothDevice bluetoothDevice) {
```

```
        if (mConnectionStatusTextViewLeft != null) {
            Log.d(TAG, String.format("%s：機器の接続が解除されました",
bluetoothDevice.getName()));
        }
    }
};
```

- 作成した `OrpheCoreCallback` と対応する取り付け位置（`OrpheSidePosition.leftPlantar` など）を指定して `Orphe` オブジェクトを作成します。

```
mOrphe = new Orphe(this, mOrpheCoreCallbackLeft,
OrpheSidePosition.leftPlantar);
```

- 作成した `Orphe` オブジェクトの `startScan` を呼び出します。

```
mOrphe.startScan();
```

- ORPHE CORE を振り光らせます。
  - 光った場合はアドバタイズしている状態になります。
- `OrpheCoreCallback` の `onScan` に見つかったアドバタイズ中の ORPHE CORE の `BluetoothDevice` オブジェクトが渡されます。
- `OrpheCoreCallback` の `onScan` で渡された `BluetoothDevice` を `Orphe` オブジェクトの `connect` に渡すことで接続されます。 `java mOrphe.connect(mBluetoothDevice);`
  - `OrpheScanedMeta` の `deviceId` を参照することでデバイス ID を取得できます。
- 接続後 `OrpheCoreCallback` の `onConnect` のコールバックが呼び出されます。
- センサー値の取得に関して
  - `OrpheCoreSensorConfig` で INSOLE と同じ 3 つの受信方式を指定できます。

```
OrpheCoreSensorConfig config = new OrpheCoreSensorConfig(
        OrpheSensorReceiveMode.fifo
);
mOrphe = new Orphe(
        this,
        mOrpheCoreCallback,
        OrpheSidePosition.leftInstep,
        config
);
```

    - `realtime`: リアルタイム Notify
    - `request`: アプリが指定した範囲だけを手動取得
    - `fifo`: SDK が carry-over 方式で継続取得と欠損回収を自動実行

    `request` / `fifo` は Android 13（API 33）以上で利用できます。Android 8.0（API 26）から Android 12L（API 32）までは `realtime` を使用してください。
  - `request` で手動取得する場合は、接続後にリクエストを送ります。1 範囲なら `requestSensorValue(startSerialNumber, length)`、複数範囲なら `requestSensorValue(OrpheValueRequest[])`を使用できます（最大 30 範囲）。

```
mOrphe.requestSensorValue(1200, 100);
```

  - `requestLatestSensorValue` を呼び出すことで最新のセンサー値を取得することが可能です。

```
mOrphe.requestLatestSensorValue();
// もしくは
mOrphe.requestLatestSensorValue(100);
```

    - `requestLatestSensorValue()`は `requestLatestSensorValue(100)`を呼び出し、100 シリアル分を取得します。
    - `length` のパラメーターを指定した場合は、**センサー値の最終取得時刻から予想されるシリアル番号から length 件**を取得します。
      - 取得件数が少なすぎると時間のズレ等でうまく取得できない可能性があるので 100 件程度までは数を増やしてください。
    - 最新シリアル番号をまだ取得していない場合、最初の呼び出しでは現在のシリアル番号だけを要求して終了します。`OrpheCoreCallback.gotCurrentSerialNumber`が呼ばれた後に、もう一度呼び出してください。
  - `requestSensorValue` を呼び出すことで自由にデバイス内に一時保存されているセンサー値を取得することができます。
    - `OrpheValueRequest` に最初のシリアル番号とそこから取得する件数を指定してパラメータに渡してください。（最大 30 種類リクエストを送ることが可能）
  - 新しい手動リクエストはデバイス上の古いリクエストを置き換えます。常時取得には、要求を直列化する `fifo` を使用してください。
- リクエストされたセンサー値は Notify で送信され、SDK がクオータニオンと重力成分を計算してから `OrpheCoreCallback` の `gotSensorValues` に渡されます。
  - Euler 角の計算と公開フィールドは一時的に無効化されています。
  - 200Hz では、加速度とジャイロを含む IMU センサー値を 1 シリアル番号につき 8 時点分取得します。シリアル番号 1 増えるごとのインターバルは理論上 40ms です。現行実装では Request / FIFO で取得した 8 時点の `startTime` / `endTime` は 5ms 間隔にならないため、時点間隔の算出には使用しないでください。
  - `startTime`、`endTime`、`receivedAt` は epoch ミリ秒です。秒へ変換する場合は `1000.0` で割ります。
- `fifo` の既定値は CORE / INSOLE 共通で、200ms ごと・最大 200 シリアル・応答待ち 5 秒・carry-over 上限 100 シリアル・リングバッファ安全上限 1,500 シリアルです。Android 版は応答を 1 件以上受信した後の無通信が 250ms 続くと早期に要求を終了します。BLE タイムアウトは次回要求へ持ち越し、デバイス内に対象シリアルが残っておらず FW から `noData` が返った場合だけ、`OrpheCoreCallback.sensorValueIsNotFound`（INSOLE では `OrpheInsoleCallback.sensorValueIsNotFound`）が番号ごとに呼ばれます。
- CORE の FIFO でも INSOLE と同様に、`OrpheSensorValueUpdate` から今回差分と再計算済み全値を取得できます。既存の配列版コールバックは変更不要です。

```
OrpheCoreCallback callback = new OrpheCoreCallback() {
    @Override
    public void gotSensorValues(OrpheSensorValueUpdate update) {
        OrpheSensorValue[] deltaValues = update.getDeltaValues();
        OrpheSensorValue[] allValues = update.getAllValues();
    }
};

OrpheSensorValue[] currentValues = mOrphe.getFifoValues();
```

- SDK 内の姿勢計算は `orphe_insole` と同じ Madgwick 6 軸フィルタ（初期姿勢 `[w,x,y,z] = [1,0,0,0]`、`beta = 0.1`、`dt = 5ms`）を使用します。加速度・ジャイロの X/Y/Z は BLE 復号値を並べ替え・符号反転せず使用し、ジャイロだけ dps から rad/s へ変換します。磁気センサーを使わないため、roll/pitch は重力で補正されますが yaw には時間経過によるドリフトがあります。
- INSOLE のジャイロ物理値は、LSM6DSOX のデータシート感度を用いて `生値[LSB] × 感度[dps/LSB]` で換算します。フルスケールを `生値 ÷ 32768 × レンジ` で割る換算は使用しません（データシート感度に対して約 12.8% 小さい値になるため）。

OrpheGyroRange	レンジ	感度[dps/LSB]
range250	±250dps	0.00875
range500	±500dps	0.0175
range1000	±1000dps	0.035
range2000	±2000dps	0.07

SDK 0.6.1 以前で記録した過去データは、`gyroX` / `gyroY` / `gyroZ` に補正係数 `1.14688`（全レンジ共通）を掛けることで補正できます。加速度・圧力・`realtime` 100Hz のデバイス算出クオータニオンは影響を受けません。`request` / `fifo`（200Hz）で SDK が算出するクオータニオンは、ジャイロを入力に使うため値が変化します。

- またセンサー値の Notify が有効になり、`OrpheCoreCallback` の `gotSensorValues` に各 Notify ごとで送信されたセンサー値が渡されます。（1 度の Notify で最大 4 つのセンサー値が渡されます）
  - Notify は 50Hz で送られており 4 つのセンサー値を送ることで最大 200Hz のセンサー値を取得することができます。
  - `startTime`、`endTime`、`receivedAt` は epoch ミリ秒です。秒へ変換する場合は `1000.0` で割ります。
- 作成した `Orphe` オブジェクトの `disconnect` を呼び出すことで切断します。

```
mOrphe.disconnect();
```

- 作成した `Orphe` オブジェクトの `status` で現在の接続ステータスを把握することが可能です。
- 作成した `Orphe` オブジェクトの `getDeviceInfo` を呼び出すことでバッテリー情報を含む ORPHE CORE の情報を取得することができます。取得した値は `OrpheCoreCallback` の `gotDeviceInfo` に渡されます。

```
mOrphe.getDeviceInfo();
```